

Módulo 7 – Capítulo 02 – Sección 01

Chain-of-Thought (CoT): pensamiento paso a paso y sus variantes

El capítulo anterior estableció que el razonamiento es el componente del agente que decide qué herramienta invocar, en qué orden, y con qué parámetros. Esta sección y las cuatro que siguen examinan las técnicas concretas que hacen ese razonamiento más preciso. La pregunta central no es teórica: ¿qué mecanismos de prompting y de arquitectura aumentan la probabilidad de que el agente tome la decisión correcta en cada paso del ciclo?

Chain-of-Thought (CoT) es la técnica de prompting que responde esa pregunta de la manera más simple: instruir al modelo a generar pasos de razonamiento intermedios explícitos antes de producir la respuesta o la acción final, en lugar de mapear directamente el input al output. La intuición es directa: así como un humano que resuelve un problema matemático complejo genera borradores y calcula parcialmente antes de dar la respuesta final, un LLM produce mejores resultados cuando "distribuye" su razonamiento en múltiples tokens intermedios en lugar de comprimirlo en una sola predicción.

La evidencia empírica es sólida. Wei et al. (2022) en "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models" (NeurIPS) demostraron mejoras de 40-50% en benchmarks de aritmética (GSM8K) y razonamiento de sentido común (StrategyQA) para modelos de más de 100B parámetros. El resultado más importante del paper no fue la mejora en sí, sino el mecanismo: CoT no hace al modelo más inteligente; le permite usar mejor la capacidad de razonamiento que ya tiene. La carga computacional de la inferencia se distribuye a través de tokens intermedios explícitos en lugar de comprimirse en una única predicción. Este insight es operativamente valioso: si el agente toma decisiones incorrectas, la primera intervención a intentar es forzar más razonamiento explícito antes de la decisión, no cambiar el modelo.

Las variantes principales de CoT tienen diferentes trade-offs en costo y facilidad de implementación. **Zero-Shot CoT** agrega simplemente la instrucción "Let's think step by step" o equivalentes al final del prompt, activando el modo de razonamiento en modelos grandes sin necesidad de ejemplos. Es la opción de menor costo de implementación y suficiente para muchos casos. **Few-Shot CoT** provee 3-8 ejemplos completos de (pregunta + cadena de razonamiento + respuesta) en el system prompt, lo que mejora la coherencia y el formato del razonamiento generado; es especialmente efectivo cuando el dominio requiere un estilo de razonamiento específico que el modelo no adopta naturalmente. **Auto-CoT** genera automáticamente los ejemplos de razonamiento usando el propio LLM en una primera pasada, reduciendo el trabajo manual de crear ejemplos de calidad; útil cuando el dominio cambia frecuentemente.

En el contexto agéntico, CoT es el mecanismo interno de razonamiento que precede cada decisión de qué herramienta invocar. En frameworks como LangGraph o en el workflow `tool_use` de Anthropic, el `Thought` generado antes de cada `Action` es CoT en acción: el agente articula por qué necesita información sobre el tema X, por qué la herramienta de búsqueda web es más adecuada que la de búsqueda en base de datos, y qué query específica formulará. Esta articulación no es decorativa: determina directamente la precisión de la acción siguiente. Hay una advertencia importante, sin embargo: el razonamiento generado puede no reflejar el proceso real de inferencia del modelo. Estudios de faithfulness muestran que los modelos pueden llegar a la respuesta correcta con razonamientos post-hoc que no corresponden al proceso que realmente produjo la predicción. Esto significa que CoT mejorar el resultado final pero no garantiza que el razonamiento visible sea la verdadera explicación de ese resultado.

Conceptos clave

- **Zero-Shot CoT:** agregar la instrucción "Let's think step by step" activa el razonamiento explícito sin necesidad de ejemplos; el punto de partida para cualquier sistema agéntico
- **Few-Shot CoT:** proveer 3-8 ejemplos de (pregunta + cadena de razonamiento + respuesta) en el system prompt mejora la coherencia y el formato del razonamiento en dominios especializados
- **Scratchpad:** espacio de texto interno donde el agente genera su razonamiento antes de tomar una decisión; en Claude, ocurre en bloques `<thinking>` separados del output final visible al usuario
- **Faithfulness del razonamiento:** el razonamiento generado puede ser post-hoc y no corresponder al proceso real de inferencia; los resultados mejoran con CoT aunque el razonamiento visible no sea causalmente responsable de la mejora
- **CoT en sistemas agénticos:** en LangGraph y ReAct, el CoT ocurre antes de cada llamada a herramienta, permitiendo al agente articular explícitamente por qué invoca una herramienta específica y con qué parámetros

Principio rector

Chain-of-Thought no hace al modelo más inteligente; le permite usar mejor la capacidad de razonamiento que ya tiene, distribuyendo la carga computacional de inferencia a través de tokens intermedios explícitos en lugar de comprimirla en una sola predicción. Cuando un agente toma decisiones incorrectas, forzar más razonamiento explícito es siempre el primer experimento a intentar.

La siguiente sección examina ReAct, el patrón que extiende CoT cerrando el bucle entre razonamiento y mundo real: en lugar de razonar en el vacío, el agente razona sobre evidencia real obtenida de herramientas.

Módulo 7 – Capítulo 02 – Sección 02

ReAct (Reason + Act): intercalar razonamiento con acciones externas

CoT mejora el razonamiento, pero razonar en el vacío tiene un límite fundamental: el modelo solo puede razonar sobre lo que ya sabe. Para tareas que requieren información actualizada, verificación factual, o acceso a datos externos, el razonamiento interno no es suficiente. ReAct es el patrón que resuelve esta limitación cerrando el bucle entre pensamiento y mundo real: el agente alterna explícitamente entre generar razonamiento y ejecutar acciones, usando los resultados de esas acciones para refinar su razonamiento en el paso siguiente.

ReAct fue presentado por Yao et al. (2022) en "ReAct: Synergizing Reasoning and Acting in Language Models" y demostró que la alternancia entre razonamiento (Thought) e invocación de acciones externas (Act) con retroalimentación de observaciones (Observation) produce resultados significativamente mejores que CoT puro en tareas que requieren acceso a información externa. La intuición del experimento es directa: en HotpotQA (un benchmark de preguntas que requieren múltiples pasos de búsqueda), ReAct supera a CoT porque puede corregir errores de razonamiento basándose en evidencia real en lugar de en inferencias. Si el agente busca "capital de Australia" y el resultado dice "Canberra", esa observación puede corregir el razonamiento previo que incorrectamente asumía "Sídney".

El formato Thought-Action-Observation que define el patrón ReAct opera de la siguiente manera en la práctica: el LLM genera primero un `Thought` en texto libre que articula el razonamiento sobre el estado actual y la siguiente acción a tomar ("Necesito verificar cuándo se fundó la empresa X para calcular su antigüedad. Usaré la búsqueda web con la query específica"). Luego genera un `Action` que especifica la herramienta y los argumentos (`search_web(query="Company X founding year")`). El sistema ejecuta la herramienta y devuelve el resultado como `Observation` ("Company X was founded in 1998 by..."). Este triplete Thought-Action-Observation se acumula en el contexto, y el LLM genera el siguiente Thought basándose en toda la evidencia acumulada. El ciclo continúa hasta que el agente tiene suficiente información para dar la respuesta final.

Este patrón es la base arquitectónica de la mayoría de frameworks agénticos modernos. LangChain AgentExecutor implementa directamente el formato Thought-Action-Observation. LangGraph con nodos de herramientas es una implementación de ReAct en forma de grafo de estado. El workflow `tool_use` de Anthropic, donde el modelo alterna entre generar bloques `text` y bloques `tool_use`, es ReAct bajo la nomenclatura de la API. OpenAI GPT-4o con function calling implementa el mismo patrón aunque no use la terminología explícita. La ubicuidad de ReAct en la práctica refleja su efectividad como patrón base para agentes que necesitan información del mundo real.

Hay un costo operativo que debe considerarse en el diseño. Cada iteración ReAct añade al contexto el Thought (50-200 tokens), la Action (20-100 tokens), y la Observation (variable, puede ser 500-2000 tokens para resultados de búsqueda web). En tareas de 10 pasos, esto puede acumular 5K-15K tokens solo en trazas de razonamiento e interacción con herramientas, antes de contabilizar el system prompt y el historial de la conversación. Esto tiene consecuencias en costo (más tokens = mayor costo de API) y en calidad (contextos más largos pueden sufrir del "lost-in-the-middle problem" que se desarrolla en el Capítulo 04). El diseño de cuánto razonamiento explicitar y cuánta observación incluir en cada paso es una variable de diseño que afecta tanto la calidad como el costo.

Aspectos técnicos

- **Formato Thought-Action-Observation:** cada iteración del bucle acumula en el contexto el razonamiento explícito, la acción invocada y su resultado; este historial guía las decisiones de acción en todos los pasos posteriores
- **Grounding externo:** a diferencia de CoT puro, ReAct puede corregir errores de razonamiento basándose en evidencia real obtenida de herramientas, reduciendo alucinaciones en tareas que requieren información factual actualizada
- **Action space definition:** el conjunto de acciones disponibles (herramientas con nombre, descripción y schema) se inyecta en el system prompt; descripciones ambiguas degradan directamente la calidad de las decisiones de Action
- **Interleaving obligatorio:** el razonamiento y la acción deben alternarse en el mismo hilo de generación del LLM; separar ambas fases en llamadas distintas al modelo rompe la coherencia del razonamiento porque el contexto de la fase Think no está disponible en la fase Act
- **Overhead de tokens:** tareas de 10 pasos pueden consumir 5K-15K tokens en trazas de razonamiento + observaciones; este costo debe considerarse en el modelo de costos del agente antes del despliegue

Nota del Arquitecto: El error más común con ReAct en producción es no limitar el tamaño de las Observations. Una búsqueda web que devuelve 10 resultados completos con 500 palabras cada uno puede añadir 5K tokens al contexto en un solo paso. La solución es doble: configurar la herramienta para devolver solo los primeros 200 caracteres por resultado, y añadir una instrucción explícita en el system prompt para que el agente sepa que puede invocar la herramienta nuevamente con una query más específica si los snippets no son suficientes.

Para recordar

ReAct transforma al LLM de un sistema que razona en el vacío a uno que razona sobre evidencia real: cada observación de herramienta es información nueva que puede confirmar, refutar o redirigir el plan del agente. Esta capacidad de corrección basada en evidencia es la diferencia entre un agente que alucina con confianza y uno que verifica antes de responder.

La siguiente sección examina Tree of Thoughts, una extensión de ReAct que en lugar de seguir un camino lineal de razonamiento, explora múltiples caminos simultáneamente para encontrar la estrategia óptima en problemas de planificación complejos.

Módulo 7 – Capítulo 02 – Sección 03

Tree of Thoughts: exploración de múltiples caminos de razonamiento

Tanto CoT como ReAct siguen un camino de razonamiento lineal: el agente genera un pensamiento, toma una acción, observa el resultado, y avanza. Esta linealidad funciona bien cuando la dirección correcta es evidente en cada paso, pero falla en una clase importante de problemas: aquellos donde la decisión correcta en el paso 2 depende de anticipar lo que ocurrirá en el paso 5, o donde un error de elección temprana invalida irrecuperablemente todos los pasos posteriores. Para estos problemas, la exploración de múltiples caminos alternativos de razonamiento antes de comprometerse con uno es la estrategia correcta.

Tree of Thoughts (ToT), propuesto por Yao et al. (2023) en "Tree of Thoughts: Deliberate Problem Solving with Large Language Models", extiende CoT de un proceso lineal a una búsqueda en árbol: en lugar de generar un único "pensamiento siguiente" en cada nodo, el modelo genera múltiples pensamientos candidatos (típicamente $k=3-5$), evalúa el valor de cada uno, y expande los más prometedores. El resultado es un árbol donde cada nivel representa un estado de avance hacia la solución y cada nodo es un pensamiento candidato; el camino de la raíz hasta una hoja es una estrategia de resolución completa. La búsqueda puede seguir dos estrategias: Breadth-First Search (BFS), que expande todos los nodos de un nivel antes de avanzar al siguiente y garantiza encontrar la solución óptima pero requiere mantener múltiples ramas activas simultáneamente; o Depth-First Search (DFS) con backtracking, que sigue un camino hasta el fallo y retrocede para explorar alternativas, más eficiente en memoria pero susceptible a mínimos locales sin un límite de profundidad adecuado.

El evaluador de nodos es el componente más crítico de ToT y el que más varía en implementaciones reales. En la formulación original del paper, el propio LLM evalúa cada pensamiento candidato con un prompt separado que pide una puntuación ("¿cuán prometedor es este camino para resolver el problema?"). En implementaciones más sofisticadas, el evaluador puede ser una función heurística programática (para problemas con métricas objetivas, como si el código parcialmente escrito compila), una red de valor entrenada para el dominio específico, o un LLM diferente especializado en evaluación. La calidad del evaluador determina directamente la utilidad de ToT: un evaluador que no distingue bien entre caminos prometedores y no prometedores convierte la búsqueda en árbol en una exploración aleatoria costosa.

En la práctica de la ingeniería de agentes, ToT tiene un caso de uso específico y acotado: es adecuado para la **fase de planificación inicial** de una tarea compleja, no para el bucle de ejecución paso a paso. Usar ToT para decidir la estrategia general de resolución de un problema de programación complejo — explorar si conviene un enfoque iterativo vs recursivo, si se debe empezar por la interfaz o la lógica

interna, qué estructura de datos es más apropiada— tiene sentido porque el costo de esa exploración (5-20 llamadas al LLM) es insignificante comparado con el costo de ejecutar la estrategia incorrecta durante muchas iteraciones. Usar ToT en cada iteración del bucle agéntico, por el contrario, es prohibitivo: 20 llamadas al LLM por paso en una cadena de 15 pasos resulta en 300 llamadas por tarea, con la latencia y el costo correspondientes.

El costo computacional de ToT es real y debe planificarse. Una exploración con $k=3$ pensamientos candidatos y profundidad $d=4$ puede requerir hasta $3^4 = 81$ evaluaciones en el peor caso con BFS, cada una involucrando una llamada al LLM. En la práctica, el pruning agresivo (descartar los nodos con puntuación baja antes de expandirlos) reduce esta cifra significativamente, pero ToT sigue siendo 5-20 veces más costoso en tokens y latencia que CoT lineal para el mismo problema. Este trade-off —mayor costo por mayor calidad de razonamiento— es lo que debe motivar la decisión de usar ToT: solo cuando un error de planificación temprano es más costoso que el tiempo adicional de exploración.

Aspectos técnicos

- **Generación de pensamientos candidatos:** en cada nodo del árbol, el LLM genera k pensamientos alternativos ($k=3-5$ típicamente) que representan diferentes formas de avanzar desde el estado actual; la diversidad de los candidatos es clave para la utilidad de la búsqueda
- **Evaluador de nodos:** función que asigna un score a cada pensamiento candidato; puede ser el propio LLM (prompt de evaluación separado), una función heurística programática o una red de valor; la calidad del evaluador determina la utilidad del árbol
- **Estrategia BFS:** expande todos los nodos del nivel actual antes de avanzar; garantiza optimalidad si el evaluador es preciso, pero requiere mantener múltiples ramas activas en memoria y puede ser prohibitivo en profundidad alta
- **Estrategia DFS con backtracking:** sigue un camino hasta el fallo y retrocede; más eficiente en memoria pero requiere un límite de profundidad explícito para evitar atascarse en mínimos locales
- **Aplicabilidad en agentes:** ToT es adecuado para la fase de planificación inicial de una tarea compleja; usar ToT en cada iteración del bucle de ejecución es prohibitivo en costo y latencia

Principio rector

Tree of Thoughts intercambia latencia y costo por calidad de razonamiento: es la técnica correcta cuando un error de planificación temprano es más costoso que el tiempo adicional de exploración, y es la técnica incorrecta cuando la velocidad de respuesta es crítica o cuando la tarea no tiene dependencias fuertes entre decisiones tempranas y tardías.

La sección siguiente examina la planificación jerárquica, un enfoque que combina la estructura de ToT con la escalabilidad de los agentes especializados: descomponer el problema en subtareas antes de ejecutarlo, en lugar de explorar el espacio de soluciones durante la ejecución.

Módulo 7 – Capítulo 02 – Sección 04

Planificación jerárquica: descomposición de tareas complejas en subtareas

CoT, ReAct y ToT son técnicas que mejoran la calidad del razonamiento dentro del ciclo de ejecución del agente. La planificación jerárquica opera a un nivel diferente: en lugar de mejorar cómo el agente razona en cada paso, reorganiza la arquitectura completa del proceso separando explícitamente la fase de planificación de la fase de ejecución. Esta separación tiene consecuencias prácticas importantes que van más allá del rendimiento: facilita la paralelización, reduce el riesgo de desviación del objetivo en tareas largas, y permite revisar o aprobar el plan antes de comprometer recursos.

La planificación jerárquica es el proceso mediante el cual un agente descompone un objetivo de alto nivel en un árbol de subtareas más manejables, donde cada subtask puede ser asignada a un sub-agente especializado, ejecutada con herramientas específicas, o resuelta con una llamada LLM independiente. El patrón más básico es el **planner-executor split**: un agente planificador recibe el objetivo, genera un plan estructurado (lista ordenada de subtareas, JSON con dependencias, o grafo de ejecución), y un agente ejecutor —o varios ejecutores especializados— implementan ese plan paso a paso. Esta separación de responsabilidades aparece en implementaciones concretas: el patrón `plan-and-execute` de LangChain, el LLM Compiler descrito por Khattab et al. (2023), y el modo `Process.hierarchical` de CrewAI donde un agente manager orquesta la distribución de subtareas.

El beneficio más inmediato de la planificación jerárquica es la **reducción del goal drift**: en tareas largas sin un plan explícito, el agente puede desviarse gradualmente de su objetivo original porque en cada paso optimiza localmente —elige la mejor acción dado el estado actual— sin verificar si esa acción contribuye al objetivo de nivel superior. Con un plan explícito al que el ejecutor puede referirse, cada acción puede verificarse: ¿este paso contribuye a completar la subtask N? ¿la subtask N es necesaria para el objetivo global? Esta propiedad de verificabilidad hace que los sistemas con planificación jerárquica sean significativamente más auditables que los agentes puramente reactivos.

La **paralelización de subtareas** es el segundo beneficio operativo importante. Cuando el plan identifica subtareas sin dependencias entre sí —por ejemplo, analizar simultáneamente tres fuentes de datos independientes—, esas subtareas pueden ejecutarse concurrentemente usando `asyncio.gather()` en Python. LangGraph implementa esto con nodos paralelos en el grafo de estado: en lugar de ejecutar los nodos A, B y C en secuencia, el grafo los inicia simultáneamente y espera a que todos terminen antes de avanzar al nodo agregador. En una tarea de análisis competitivo que investiga cinco empresas, la

paralelización puede reducir la latencia total de 5 minutos (secuencial) a poco más de 1 minuto (paralelo), asumiendo que el bottleneck es la latencia de las herramientas de búsqueda.

La **re-planificación dinámica** es el aspecto más sofisticado de este patrón y el que más frecuentemente se omite en implementaciones iniciales. Cuando una subtask falla o produce resultados que cambian los supuestos del plan original, el agente debe poder actualizar el plan en lugar de continuar ciegamente ejecutando subtasks que ya no son válidas. En LangGraph, esto se implementa como un nodo de evaluación del plan después de cada subtask: si el resultado es inesperado, el nodo devuelve control al planificador con el contexto actualizado para que genere un plan revisado. Sin este mecanismo, un plan que falla en la subtask 2 de 8 lleva al agente a ejecutar las subtasks 3-8 sobre bases incorrectas.

La granularidad de la descomposición es una variable de diseño crítica. Subtasks demasiado granulares —cada acción individual como una subtask— generan overhead de coordinación sin beneficio: el planificador pasa más tiempo administrando el plan que el ejecutor implementándolo. Subtasks demasiado grandes recrean el problema original: una subtask de "analizar el mercado" sin especificación adicional no ayuda al ejecutor más que el objetivo original. El tamaño óptimo en la práctica es aquel que puede completarse con 2-5 acciones de herramienta y que produce un entregable intermedio verificable: un documento resumido, un conjunto de datos estructurado, o una decisión específica que alimenta la subtask siguiente.

Aspectos técnicos

- **Planner-Executor split:** el planificador genera un plan estructurado (JSON, lista numerada, grafo de dependencias) y el ejecutor lo implementa; separa la responsabilidad de estrategia de la responsabilidad de ejecución, permitiendo optimizar cada componente de forma independiente
- **Descomposición recursiva:** subtasks complejas pueden descomponerse en sub-subtasks; la profundidad máxima del árbol debe limitarse explícitamente para evitar over-decomposition con overhead de coordinación creciente
- **Paralelización de subtasks independientes:** subtasks sin dependencias entre sí pueden ejecutarse concurrentemente con `asyncio.gather()`; LangGraph implementa esto con nodos paralelos en el grafo de estado; puede reducir la latencia total de forma drástica en workflows con muchas subtasks independientes
- **Re-planificación dinámica:** nodo de evaluación del plan después de cada subtask; si el resultado difiere de lo esperado, el planificador puede generar un plan revisado con el contexto actualizado; sin este mecanismo el sistema falla silenciosamente cuando los supuestos del plan original no se cumplen
- **Granularidad óptima de subtasks:** cada subtask debe poder completarse con 2-5 acciones de herramienta y producir un entregable intermedio verificable; ni tan granular que el overhead de coordinación domine, ni tan grande que no aporte más claridad que el objetivo original

Nota del Arquitecto: El error más común en planificación jerárquica es que el planificador genera un plan demasiado rígido que el ejecutor intenta seguir literalmente incluso cuando los resultados intermedios contradicen los supuestos del plan. La solución es diseñar el sistema prompt del ejecutor para que comunique activamente al planificador cuando una subtarea produce resultados inesperados, y diseñar el planificador para incorporar esa información antes de continuar. Un plan jerárquico debe ser una guía adaptable, no una especificación inmutable.

Principio rector

La planificación jerárquica no elimina la incertidumbre de la ejecución, pero la hace manejable: al separar la estrategia de la táctica, el agente puede detectar y corregir desviaciones localmente sin perder el objetivo global, y puede aprovechar el paralelismo natural de las subtarefas independientes para reducir la latencia total del sistema.

La sección siguiente examina la verificación y autocorrección, el complemento necesario a la planificación: una vez que el agente ejecuta un paso, ¿cómo verifica que el resultado es correcto antes de usarlo como base para el siguiente?

Módulo 7 – Capítulo 02 – Sección 05

Verificación y autocorrección: el agente evalúa sus propios pasos

Planificar bien y razonar bien no garantizan que cada paso del ciclo produzca el resultado correcto. Los LLMs cometen errores: alucinan datos, interpretan incorrectamente observaciones de herramientas, seleccionan herramientas inadecuadas para ciertos casos de borde, y generan código que no funciona. La autocorrección agéntica es el mecanismo que cierra ese gap: en lugar de propagar cada error al paso siguiente, el sistema detecta el problema, evalúa su gravedad, y toma la acción correctiva antes de continuar. La clave es que esta verificación ocurra dentro del ciclo de ejecución, sin requerir intervención humana para cada paso, pero también sin dejar al sistema ciego ante sus propios errores.

La forma más directa de autocorrección es la **verificación externa**: ejecutar el resultado del agente en el entorno real y observar si produce el efecto esperado. Un agente que genera código Python puede ejecutarlo en un sandbox E2B y verificar si compila y si los tests pasan. Un agente que genera una URL puede hacer una llamada HTTP y verificar que devuelve HTTP 200. Un agente que genera un JSON puede validarlo contra un schema Pydantic o JSON Schema. Esta verificación es binaria, programática, y no introduce la incertidumbre adicional de una segunda llamada al LLM. Siempre que los criterios de corrección sean formalizables, la verificación externa es la forma más confiable de autocorrección y debe preferirse sobre cualquier evaluación basada en LLM.

Cuando los criterios de corrección son semánticos y no formalizables, la autocorrección debe involucrar otro paso de inferencia. El **patrón de reflexión** (reflection pattern) es el más directo: el mismo agente que generó la respuesta recibe un prompt adicional que le pide identificar posibles errores y autocorregirse. "Revisa tu respuesta anterior. ¿Hay afirmaciones que podrían ser incorrectas? ¿El razonamiento es consistente con las observaciones disponibles? Si encuentras errores, corrígelos." Estudios experimentales muestran que esta auto-revisión mejora los resultados en 10-30% en tareas de codificación y razonamiento matemático. La limitación es igualmente documentada: el modelo puede cometer el mismo error tanto en la generación como en la evaluación si el error proviene de un sesgo sistemático del modelo o de información incorrecta en el contexto.

El **patrón LLM-as-judge** mitiga esta limitación usando un segundo modelo o un segundo prompt radicalmente diferente para evaluar la respuesta del primer agente. El juez recibe la tarea original, la respuesta del agente, y un rubric de evaluación explícito, y devuelve un score más justificación textual de sus observaciones. Usar GPT-4o como juez para evaluar outputs de modelos menores, o usar Claude 3.5 Sonnet como juez para outputs generados por el mismo Claude con un prompt diferente, puede detectar errores que la auto-reflexión pierde. El costo es real: cada evaluación es una llamada adicional al LLM con toda la latencia y costo asociados.

La **self-consistency** es una técnica diferente que evita el problema de la evaluación: en lugar de evaluar una respuesta, genera múltiples respuestas independientes para el mismo problema y aplica voting para seleccionar la más frecuente. Generar $k=5$ respuestas y elegir por mayoría es efectivo para problemas con respuesta única discreta (clasificaciones, cálculos numéricos, selección de categorías) donde la varianza del modelo puede controlarse con temperatura moderada. Su limitación es el costo: $k=5$ respuestas significa 5 llamadas al LLM con el costo de tokens completo de cada una.

Para todos los patrones de autocorrección, la política de reintentos es tan importante como el mecanismo de verificación. Sin un límite de intentos, la autocorrección puede generar bucles donde el agente corrige indefinidamente sin mejorar, consumiendo tokens y latencia de forma proporcional. El diseño estándar es `max_retries=3` con escalado al humano si se supera ese límite. Cuando una herramienta falla repetidamente o el agente no puede producir un resultado válido después de tres intentos, la decisión correcta es comunicar el fallo explícitamente —con diagnóstico de qué fue lo que no funcionó— en lugar de degradar silenciosamente la calidad del resultado.

Puntos críticos

- **Verificación externa como primera opción:** ejecutar el resultado y verificar el efecto real es siempre la forma más confiable de autocorrección; preferir sobre LLM-as-judge cuando los criterios de éxito son formalizables (el código compila, el JSON valida, la URL responde)
- **Reflection pattern:** el agente revisa su propia respuesta con un prompt de evaluación diferenciado; mejora resultados en 10-30% en tareas de codificación y razonamiento, pero falla ante sesgos sistemáticos del modelo
- **LLM-as-judge:** un segundo modelo o prompt radicalmente diferente evalúa la respuesta con un rubric explícito; más robusto que la auto-reflexión pero añade latencia y costo de una llamada adicional al LLM por cada paso evaluado
- **Self-consistency:** generar k respuestas independientes y seleccionar por majority voting; efectivo para respuestas discretas, costoso para respuestas de texto libre
- **max_retries con escalado:** limitar los intentos de autocorrección (`max_retries=3`) y escalar al humano cuando se supera; la autocorrección sin límite puede consumir más recursos que reiniciar la tarea desde el principio

Nota del Arquitecto: La tentación de usar LLM-as-judge en cada paso del ciclo para garantizar calidad es real, pero el costo operativo puede volverse mayor que el beneficio. En producción, aplicar verificación por LLM-judge selectivamente: solo en los pasos donde el error tiene mayor impacto en el resultado final o donde la verificación programática no es factible. Para los demás pasos, confiar en la verificación programática y reservar los recursos de evaluación para donde más importan.

Buena práctica

Priorizar verificación externa sobre autocorrección por LLM siempre que sea posible: ejecutar el resultado en un sandbox o verificarlo contra un schema es una verificación de verdad objetiva, mientras que la autocorrección por LLM es solo otra inferencia probabilística sujeta a los mismos sesgos y limitaciones del modelo que generó el resultado original.

La sección siguiente cierra el capítulo sobre razonamiento con la síntesis que conecta CoT, ReAct, ToT, planificación jerárquica y autocorrección en un marco unificado de criterios de selección para el ingeniero que debe elegir qué técnica usar en cada situación.

Módulo 7 – Capítulo 02 – Sección 06

Cierre: la calidad del razonamiento determina la calidad de las acciones del agente

El capítulo de razonamiento y planificación establece una verdad operativa central: en sistemas agénticos, el LLM no es solo el motor de generación de texto sino el módulo de toma de decisiones que determina qué acción ejecutar, en qué orden y con qué parámetros. La calidad de esas decisiones —y por tanto la calidad de los resultados del agente— está directamente limitada por la calidad del razonamiento que las precede. Un agente cuyo razonamiento es opaco, implícito o apresurado producirá acciones incorrectas con mayor frecuencia, y esos errores se amplificarán en cadenas largas hasta hacer la tarea fallida.

Las cuatro técnicas cubiertas en este capítulo —CoT, ReAct, ToT y planificación jerárquica— no son alternativas entre sí sino herramientas con diferentes perfiles de costo-beneficio que se aplican a diferentes tipos de problemas. El error más común es elegir entre ellas basándose en popularidad o en lo que se usó en el último proyecto: la elección correcta depende de la estructura del problema, el perfil de latencia requerido, y el presupuesto de tokens disponible. La tabla siguiente proporciona una guía de referencia rápida para esa decisión.

Tabla comparativa de técnicas de razonamiento agéntico

Técnica	Overhead de tokens	Latencia relativa	Caso de uso ideal	Limitación principal	Frameworks con soporte nativo
Chain-of-Thought (CoT)	Bajo (+50-200 tokens por paso)	Mínima (~0ms adicional)	Razonamiento sobre información ya disponible en el contexto; decisiones de qué herramienta usar	No puede corregir errores con información nueva; faithfulness del razonamiento no garantizada	Cualquier LLM con system prompt
ReAct	Medio (+100-500 tokens por iteración incluyendo observaciones)	Igual a CoT + latencia de herramientas	Tareas que requieren información externa en tiempo real; verificación factual; búsqueda iterativa	Overhead de tokens por observación; latencia acumulada en cadenas largas	LangGraph, LangChain AgentExecutor, AutoGen, API nativa de Anthropic/OpenAI
Tree of Thoughts (ToT)	Alto ($k \times d$ llamadas adicionales al LLM, donde k =candidatos y d =profundidad)	Alta (5-20x respecto a CoT)	Planificación inicial de tareas con fuertes dependencias entre decisiones; problemas donde un error temprano invalida el resto	Prohibitivo para uso en cada iteración del ciclo; requiere un evaluador de calidad para ser útil	Implementación manual; no nativo en frameworks estándar
Planificación jerárquica	Medio-Alto (una llamada de planificación adicional al inicio)	Media (overhead de planificación al inicio, ganancia de paralelismo en ejecución)	Tareas decomponibles en subtarear independientes; workflows donde se requiere aprobación del plan antes de ejecutar	Requiere re-planificación cuando los supuestos del plan original no se cumplen	LangGraph (nodos paralelos), CrewAI (Process.hierarchical), LLM Compiler
Verificación y autocorrección	Variable (o para	Variable	Pasos críticos donde el error	Fallos ante sesgos	LangGraph (nodo de evaluación),

Técnica	Overhead de tokens	Latencia relativa	Caso de uso ideal	Limitación principal	Frameworks con soporte nativo
	verificación programática; +1 llamada LLM para reflexión o judge)		tiene alto impacto; outputs que deben cumplir contratos de formato	sistemáticos del modelo; costo prohibitivo si se aplica a cada paso	LangSmith (evaluación automatizada)

Cuándo usar cada técnica: guía de decisión práctica

CoT es el default razonable para cualquier agente nuevo. Siempre actívalo, ya sea como Zero-Shot CoT ("piensa paso a paso") o como Few-Shot CoT con ejemplos del dominio. El costo es mínimo y el beneficio es consistente.

ReAct es la extensión natural de CoT cuando la tarea requiere información externa. Si el agente necesita buscar, leer, ejecutar o verificar datos del mundo real, ReAct es el patrón base a usar. Casi todos los frameworks agénticos modernos lo implementan nativamente.

ToT se justifica cuando la tarea tiene características específicas: múltiples caminos de solución con diferentes quality levels, donde la elección temprana entre ellos tiene consecuencias irreversibles en los pasos posteriores. El caso de uso más frecuente en ingeniería es la planificación inicial de una arquitectura de software o de una estrategia de investigación compleja. Fuera de ese caso de uso, el costo en tokens y latencia raramente justifica el beneficio.

Planificación jerárquica se justifica cuando la tarea tiene más de 5-7 pasos y las subtarefas son suficientemente independientes como para paralelizarse o para asignarse a agentes especializados. Si la tarea es inherentemente secuencial y sin paralelismo natural, la planificación jerárquica añade overhead sin beneficio proporcional.

Autocorrección siempre que los pasos críticos del ciclo produzcan outputs formalizables (código, JSON, URLs). Para outputs semánticos, usar LLM-judge selectivamente en los pasos de mayor impacto, no en cada paso del ciclo.

Invertir en mejorar la calidad del razonamiento —a través de prompts más estructurados, técnicas de reflexión, y evaluación de pasos intermedios— tiene mayor retorno que agregar más herramientas o aumentar el límite de iteraciones. La diferencia entre un agente que completa el 60% de las tareas y uno que completa el 90% rara vez está en las herramientas disponibles; está en la calidad del razonamiento que guía cuándo y cómo usarlas.

Principio rector

La diferencia entre un agente que completa el 60% de las tareas y uno que completa el 90% rara vez está en las herramientas disponibles; está en la calidad del razonamiento que guía cuándo y cómo usarlas.

Invertir en razonamiento antes de invertir en herramientas.

El capítulo siguiente completa el análisis del lado de las herramientas: si el razonamiento determina qué hacer, las herramientas determinan qué es posible hacer. Diseñarlas correctamente —con la misma precisión con la que se diseñan los prompts de razonamiento— es el próximo desafío de la ingeniería agéntica.

"Thinking is the hardest work there is, which is probably the reason why so few engage in it." — sentencia habitualmente atribuida a Henry Ford; independientemente de su autoría original, el principio se aplica literalmente a los sistemas agénticos: forzar al modelo a generar pensamiento explícito en tokens produce consistentemente mejores decisiones que permitirle responder de forma inmediata y sin articulación del razonamiento.